

PROJECT CONTEXT DOCUMENT

RGA

Agentic Requirement Gathering & Analysis

An AI system that turns raw, messy project documents into a human-reviewed, evidence-traceable requirements specification — the first phase of the software delivery lifecycle.

Prepared by RGA · **Version** 1.0 · **Phase** Requirements (SDLC)

Contents

- 1 Executive summary
- 2 The problem & the solution
- 3 How it works — the four phases
- 4 Inside the pipeline — the AI agents
- 5 System architecture
- 6 What it consumes (LLM usage)
- 7 Inputs & outputs
- 8 Principles & guarantees
- 9 Technology & how it runs
- 10 Where it fits & what's next

1 • Executive summary

RGA takes the pile of documents every project starts with — business requirements, discovery-call notes, email threads, backlogs — and turns them into one clean, agreed, professional **requirements specification**, in a fraction of the usual time.

A pipeline of AI agents reads the documents the way an experienced analyst would: it extracts the requirements, removes duplicates, resolves scope, flags contradictions, and checks for gaps. A person then reviews and approves everything through a simple decision-by-decision interface. Finally, RGA generates an industry-standard **IEEE-830 Software Requirements Specification (SRS)** and a **Requirements Traceability Matrix (RTM)**, ready to hand to the design team.

The outcome: what typically takes a team several days of manual effort is produced in an afternoon — with fewer errors, complete transparency, and a human in control of every decision.

2 • The problem & the solution

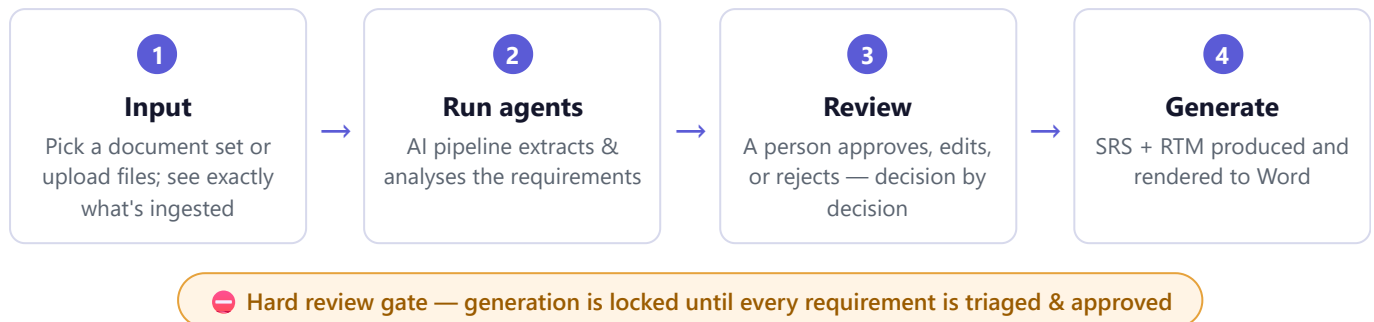
The problem

Requirements live scattered across many documents, written by different people in different styles. Turning them into a single, unambiguous, non-contradictory specification is slow and manual — and it's easy to miss things, duplicate them, or lose the link back to where a requirement actually came from. Mistakes here are expensive: they propagate into design, build, and testing.

The solution

RGA automates the heavy lifting while keeping people in charge. AI agents do the reading and first-pass analysis in seconds; a human confirms the important calls; and the output is a standards-based document where **every requirement traces back to a byte-accurate quote** from the source. Nothing is invented, and nothing reaches the document until a human approves it.

3 · How it works — the four phases



- 1 · Input.** The user selects a prepared set of project documents (or uploads their own). The system first shows the metadata of every file — name, format, size — so it's always clear what data went in.
- 2 · Run.** A background pipeline of agents reads the documents and produces a clean, analysed set of candidate requirements. Progress is shown live, step by step.
- 3 · Review.** The human control point. Instead of reading hundreds of items one by one, the reviewer works through a short list of **decisions** — each with a recommendation and its reasoning — and confirms the technology stack. Resolving one decision applies everywhere it's relevant.
- 4 · Generate.** Once the gate opens, RGA drafts the descriptive prose, assembles the SRS and RTM, and renders professional Word documents ready for the design team.

4 · Inside the pipeline — the AI agents

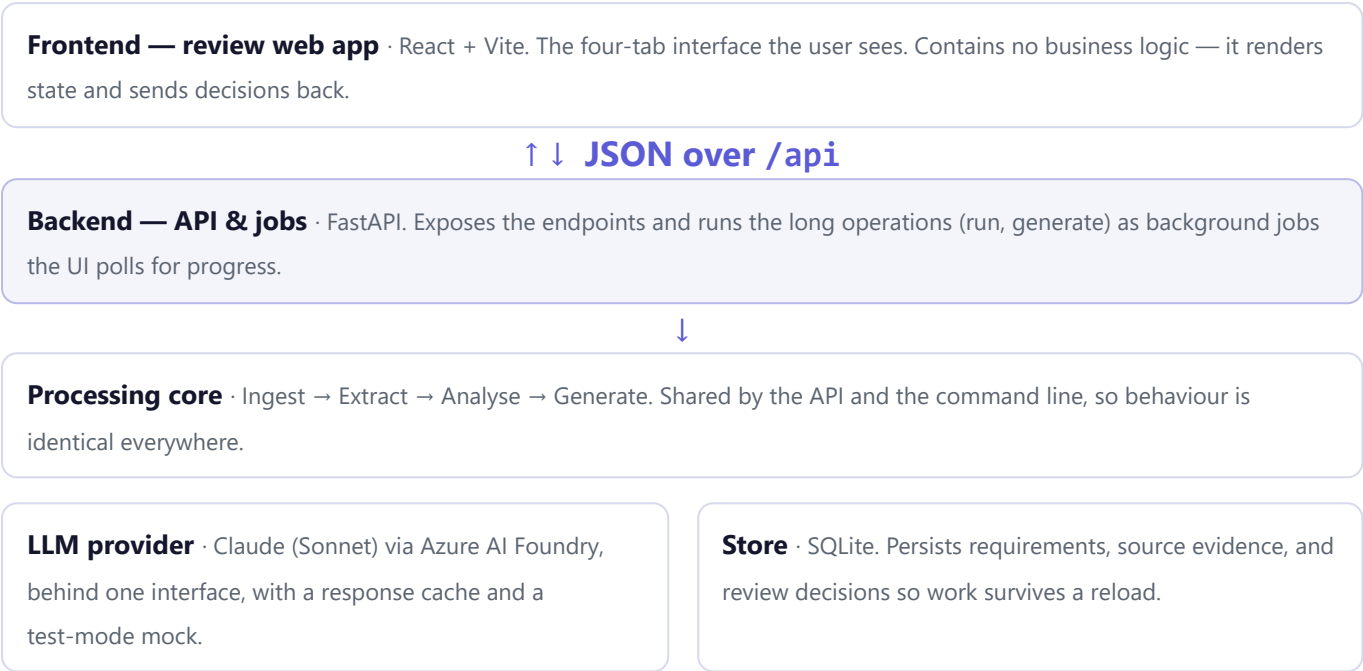
The "Run" phase is a sequence of specialised agents. Each does one job well; together they turn raw text into a trustworthy requirement set.

1. **Extraction & Critic** — read each piece of the documents and pull out candidate requirements, each backed by an exact source quote; a critic rejects anything invented or ungrounded.
2. **De-duplication** — collapse the same requirement written in different words.
3. **Consolidation** — merge everything into one canonical statement per requirement.
4. **Scope reconciliation** — drop items the sources mark out of scope for this release.
5. **Conflict detection** — surface requirements that contradict each other, for a human to resolve.
6. **Verification** — an independent second opinion on the riskiest requirements.
7. **Completeness check** — flag commonly-expected requirements the documents didn't mention.
8. **Clarity & priority** — score wording for ambiguity and assign High / Medium / Low.
9. **Technology stack** — adopt a stated stack, or propose simple, popular options per area.
10. **Auto-approve routine** — clear, confident items are auto-approved; the rest go to the human.

Everything else — grounding checks, triage, traceability, glossary, and the Word render — is **deterministic** (rule-based, no AI), which keeps the output stable and repeatable.

5 · System architecture

RGA is built as two applications that run independently and talk over a simple web API.



COMPONENT	TECHNOLOGY	ROLE
Frontend	React, Vite, TypeScript	The 4-tab review UI
Backend API	FastAPI (Python)	Endpoints + background run/generate jobs
Agents & orchestration	Python, LangGraph	Extract, analyse, verify, propose stack
LLM provider	Claude Sonnet · Azure AI Foundry	All AI calls, cached
Store	SQLite	Requirements, evidence, decisions
Generators	python-docx + templates	IEEE-830 SRS, RTM, Word renders

6 · What it consumes (LLM usage)

All AI runs on **Claude Sonnet** via Azure AI Foundry, at temperature 0 (deterministic). Cost is driven almost entirely by the **extraction** step, which scales with the size of the input documents; the analysis and generation steps add only a small, fixed number of calls.

COST CENTRE	HOW IT SCALES	APPROX. CALLS
Extraction + Critic	Per document chunk × passes	~2 per chunk (the bulk)
Analysis agents	Fixed per run	~6 calls
Verification	High-risk subset only	a handful
Generation	Fixed per generate	2 calls

Measured example. A 6-document set (≈100 evidence chunks) used about **255 AI calls** for a complete Run + Generate, on the order of **300k–400k tokens** total. **Because every response is cached, re-running the same inputs is effectively free.**

7 · Inputs & outputs

Inputs

A set of project documents in everyday formats: **.docx** **.pdf** **.txt** **.md** **.csv** — for example a BRD, a discovery-call transcript, an email thread, notes, an intake form, and a product backlog.

Outputs — the handoff pack

- SRS** — the IEEE-830 requirements specification (Word + Markdown).
- RTM** — the traceability matrix linking every requirement to its source line.
- Manifest** — a record of what was produced.

The design team consumes the SRS with a strict automated parser, so RGA guarantees a fixed, standards-compliant document structure on every run.

8 · Principles & guarantees

PRINCIPLE	WHAT IT MEANS
Human-in-the-loop	A hard gate means no un-approved content ever reaches the documents.
Full traceability	Every requirement carries a byte-accurate source quote; the RTM makes the trail explicit.
Nothing invented	The critic agent rejects any requirement not grounded in the source text.
Deterministic structure	The document's tables and sections are built from data, so the output is identical and parser-safe every run.
Repeatable & cheap	Temperature 0 plus response caching make re-runs consistent and effectively free.

9 · Technology & how it runs

The system runs as two local services: the **backend** (Python / FastAPI, serving the API and the agent pipeline) and the **frontend** (React / Vite review app). The backend uses Claude on Azure AI Foundry for the AI steps and SQLite for storage; both are started with simple, separate commands and reach each other over `http://127.0.0.1:8000`.

Full setup, endpoints, logs, and the integration contract for a combined SDLC frontend are documented in the repository's **README.md**, **ARCHITECTURE.md**, and **INTEGRATION.md**.

10 · Where it fits & what's next

RGA is the **Requirements** phase of the wider software delivery lifecycle. Its handoff pack — a clean, approved, fully traceable SRS + RTM — is the direct input to the **Design** phase, and the documents are formatted specifically so the design team's tooling can consume them automatically.

As the individual SDLC phases are combined into one unified platform, RGA plugs in through its web API: other phases call it to run the pipeline, read the requirements, and pull the finished documents — making requirements the reliable, auditable foundation for everything built on top.